port-scanner

Guia para entender, testar e modificar o código

Documentação em português da implementação refatorada. Inclui fundamentos, leitura por módulo, exemplos de manutenção e fontes numerados.

Use somente em destinos próprios ou autorizados. A apostila histórica permanece separada.

Sumário

1. O que este programa faz 3

2. Como executar, passo a passo 4

3. Como interpretar os resultados 5

4. Onde fica cada responsabilidade 6

5. Configuração: lendo config.py 7

6. Sondagens: lendo probes.py 8

7. Orquestração: lendo scanner.py 9

8. Interface e compatibilidade 10

9. Por que a estrutura facilita manutenção 10

10. Como testar com confiança 11

11. Receitas para modificar o código 12

12. PDF, fontes e reprodução 13

Código-fonte — main.py 15

Código-fonte — connection.py 16

Código-fonte — port_scanner/__init__.py 17

Código-fonte — port_scanner/__main__.py 18

Código-fonte — port_scanner/config.py 19

Código-fonte — port_scanner/probes.py 20

Código-fonte — port_scanner/scanner.py 21

Código-fonte — port_scanner/cli.py 22

port-scanner: guia para entender e modificar

Repositório oficial: https://github.com/volpszz/port-scanner. O nome público do projeto é `port-scanner`; o pacote Python usa `port_scanner`, pois nomes de módulos não usam hífen. A licença MIT e a autoria original de volpsz são preservadas.

Este é o guia da implementação refatorada em `port_scanner/`. Ele explica os fundamentos necessários, acompanha a execução, descreve cada módulo e mostra como alterar o projeto com segurança. O PDF correspondente é `docs/guia-do-projeto.pdf` e inclui o código-fonte dos módulos principais com numeração de linhas.

Não é necessário ser especialista em redes para começar. É útil conhecer variáveis, funções, condicionais e laços em Python. Os termos menos comuns são explicados ao longo do texto. Os nomes das funções permanecem em inglês para manter o contrato do código; as explicações deste guia estão em português.

A apostila `guia-redes-port-scanner.pdf` e `docs/snapshot/` são materiais históricos. Eles não foram substituídos por este guia: documentam uma edição anterior do programa.

1. O que este programa faz

Um computador pode executar vários serviços de rede. O endereço IP identifica o destino; uma porta identifica um ponto de comunicação associado a um protocolo de transporte. A porta 80/TCP e a porta 80/UDP são pontos diferentes, mesmo usando o mesmo número.

O scanner recebe um destino, uma lista de portas e os protocolos desejados. Para cada combinação, faz uma tentativa de comunicação e relata o que observou. Ele não identifica automaticamente o software por trás da porta, não procura vulnerabilidades e não prova que um serviço está seguro.

1.1 Vocabulário essencial

• **Alvo ou destino:** computador que será testado. Pode ser informado como IPv4 ou nome de máquina.

• **IPv4:** formato de endereço como `127.0.0.1`. Esta implementação não suporta IPv6.

• **Hostname:** nome que precisa ser convertido em endereço pelo resolvedor do sistema, frequentemente usando DNS.

• **Loopback:** interface de retorno ao próprio computador. Os exemplos e testes usam `127.0.0.1` para não acessar terceiros.

• **Porta:** número entre 1 e 65535 aceito pela configuração do scanner. A porta 0 é usada apenas nos servidores de teste para pedir ao sistema uma porta temporária.

• **Socket:** recurso do sistema operacional usado pelo Python para enviar e receber dados de rede.

• **Sondagem ou probe:** uma tentativa de observar o estado de uma porta. Não é uma garantia permanente.

• **Timeout:** prazo de espera de uma operação de socket. Prazo curto pode gerar resultados inconclusivos mesmo quando um serviço responde com atraso.

• **Payload:** conteúdo enviado. Em UDP, o scanner usa os bytes `b'probe'`.

• **CLI:** interface de linha de comando, incluindo os argumentos e o menu.

1.2 Uso responsável e limites

Teste somente máquinas próprias ou com autorização explícita. O programa não verifica essa autorização e pode gerar logs, alertas e carga no destino. Não é preciso executar com privilégios administrativos.

O programa testa um único endereço IPv4 por execução. Quando um nome resolve para vários endereços, apenas o endereço escolhido pelo resolvedor é usado. Não há varredura de sub-redes, execução paralela, captura de pacotes, descoberta de vulnerabilidades ou SYN scan.

As explicações de rede são conceituais, não capturas reais de tráfego. Os estados refletem uma observação naquele instante e dependem da rede, do sistema e do serviço.

2. Como executar, passo a passo

Abra um terminal na pasta do projeto. Use Python 3.10 ou superior. O scanner e seus testes usam somente a biblioteca padrão, que acompanha o Python. A geração do PDF tem dependências opcionais separadas.

2.1 Modo interativo

```
python main.py
```

O programa apresenta as opções `1. TCP`, `2. UDP` e `3. BOTH`. Digite o número e depois informe um destino autorizado. Para estudar localmente, informe `127.0.0.1`.

Uma escolha inválida repete o menu. Espaços antes e depois da escolha são ignorados. Ctrl+C ou fim da entrada encerra o programa com código 130.

A mesma execução está disponível como pacote:

```
python -m port_scanner
```

A opção `-m` pede ao Python que execute o arquivo `__main__.py` do pacote. Não é necessário instalar o projeto: execute a partir da raiz para o Python localizar a pasta `port_scanner`.

2.2 Modo com argumentos

```
python -m port_scanner 127.0.0.1 --protocol both --ports 8000,9999 --timeout 0.2
```

• `127.0.0.1` é o destino.

• `--protocol both` seleciona TCP e UDP. Outras opções são `tcp` e `udp`.

• `--ports 8000,9999` define as portas. Use vírgulas, não intervalos como `8000-8010`.

• `--timeout 0.2` define o prazo por operação de socket, em segundos. Use ponto decimal.

Com alvo explícito e sem `--protocol`, o padrão é TCP e não há menu. Sem alvo, o programa solicita o destino e, se o protocolo também não foi informado, mostra o menu.

Sem `--ports`, a lista original é `53,22,80,111,139,443,32768`, nessa ordem. O timeout padrão é `1.0` segundo. Portas e protocolos repetidos são removidos pela configuração sem reordenar os demais.

```
python -m port_scanner --help
```

O comando acima mostra as opções reais da versão executada. `python main.py` também aceita os argumentos apresentados aqui. Mensagens da interface permanecem em inglês por compatibilidade.

2.3 Prazo de uma execução

O scanner é sequencial: termina uma tentativa antes de começar a próxima. Várias portas e timeouts aumentam o tempo total. O timeout não representa o prazo de toda a varredura, não limita a resolução DNS e pode ser aplicado a mais de uma operação de socket. Não use a multiplicação de portas pelo timeout como garantia exata de duração.

3. Como interpretar os resultados

Cada resultado é impresso como `porta/protocolo: estado`. Por exemplo, a forma `8000/tcp: open` significa que a sondagem TCP conseguiu estabelecer uma conexão naquele momento. Esse texto é um exemplo de formato, não a afirmação de que há um servidor ativo no seu computador.

3.1 TCP

TCP estabelece uma conexão antes de trocar dados de aplicação. O scanner usa `connect_ex`, que devolve um código numérico do sistema operacional.

• Código `0`: a conexão foi estabelecida; resultado `open`.

• Código `ECONNREFUSED`: houve recusa explícita; resultado `closed`.

• Códigos `ETIMEDOUT`, `EAGAIN` ou `EWOULDBLOCK`: resultado `filtered/unknown`.

• Outros códigos: resultado `error: NOME_DO_ERRO`, quando o nome é conhecido.

Uma tentativa sem resposta não equivale a recusa. O nome `filtered/unknown` preserva essa incerteza; não demonstra sozinho a existência de um firewall. O scanner não envia uma requisição HTTP nem identifica o serviço que aceitou a conexão.

3.2 UDP

UDP não estabelece uma conexão como TCP. No socket UDP, `connect` apenas associa o destino e permite utilizar `send` e `recv`; não prova que a porta está aberta.

O scanner envia `b'probe'` e espera uma resposta:

• Resposta recebida, mesmo vazia: `open`.

• `ConnectionRefusedError`: `closed`, quando o sistema entrega a recusa ao processo.

• Tempo esgotado: `open|filtered`.

• Outro erro de rede: tratado como erro individual pelo orquestrador.

Um serviço pode estar aberto e ignorar o payload genérico por não ser uma mensagem válida do seu protocolo. DNS, por exemplo, exige uma consulta com estrutura própria. Por isso silêncio UDP nunca deve virar automaticamente `closed`.

3.3 Erro individual e encerramento

Um erro de rede em uma sondagem aparece como `error: ...` e não impede testar as próximas portas. Já a impossibilidade de resolver o destino encerra a execução, pois não há endereço para testar.

• Código de saída `0`: o programa terminou o fluxo, mesmo que haja resultados fechados, inconclusivos ou erros individuais.

• Código `1`: falha ao resolver o destino.

• Código `2`: argumento ou configuração inválida.

• Código `130`: Ctrl+C ou fim da entrada interativa.

Resultados, aviso de autorização e perguntas vão para `stdout`, a saída normal. Mensagens de configuração, DNS e interrupção vão para `stderr`, a saída de erro. Portanto a saída normal não é um formato de dados puro para integração automatizada; ela inclui mensagens humanas.

4. Onde fica cada responsabilidade

O objetivo da refatoração não foi criar o maior número de arquivos possível. Foi permitir encontrar a regra que você deseja alterar sem misturar entrada do usuário, rede e impressão.

```
main.py / port_scanner.__main__
              |
              v
cli.py: argumentos, menu e apresentacao
              |
              v
config.py: configuracao validada
              |
              v
scanner.py: resolve e percorre portas/protocolos
              |
              v
probes.py: faz contato com os sockets
              |
              v
scanner.py: entrega ScanResult
              |
              v
cli.py: imprime os resultados
```

• `main.py`: entrada tradicional e ponte com os nomes legados.

• `connection.py`: fachada que oferece os nomes antigos sem duplicar as implementações.

• `port_scanner/__init__.py`: identifica o pacote, sem iniciar tarefas.

• `port_scanner/__main__.py`: entrada para `python -m port_scanner`.

• `port_scanner/config.py`: padrões e validação de `ScanConfig`.

• `port_scanner/probes.py`: operações TCP/UDP e interpretação de respostas.

• `port_scanner/scanner.py`: coordenação das sondagens e estrutura de resultado.

• `port_scanner/cli.py`: entrada, saída e tradução de falhas esperadas em códigos de saída.

• `tests/`: verificações da implementação atual, separadas por camada.

• `test_connection.py`: contratos da interface anterior.

• `docs/`: guias atuais, geradores e material histórico, identificados separadamente.

5. Configuração: lendo config.py

5.1 Constantes e campos

`DEFAULT_PORTS`, `DEFAULT_TIMEOUT` e `SUPPORTED_PROTOCOLS` centralizam valores compartilhados. Evita-se copiar a mesma lista em cada módulo e esquecer de atualizar uma das cópias.

`ScanConfig` é uma classe de dados, criada com `@dataclass(frozen=True)`. A decoração faz o Python gerar métodos usuais, como o construtor. `frozen=True` impede reatribuições normais aos campos após a criação.

Os campos são `target`, `ports`, `protocols` e `timeout`. Os dois primeiros conceitos são destino e portas; `protocols` usa textos minúsculos como `('tcp', 'udp')`.

```
from port_scanner.config import ScanConfig

config = ScanConfig(
    target='127.0.0.1',
    ports=(8000, 9999),
    protocols=('tcp', 'udp'),
    timeout=0.2,
)
```

As anotações como `tuple[int, ...]` explicam os tipos esperados para pessoas, editores e ferramentas. Elas não convertem automaticamente qualquer entrada em um valor válido. Passe texto como alvo, inteiros como portas e número como timeout.

5.2 Validação no construtor

Depois de criar os campos, a dataclass chama `__post_init__`. Esse método:

1. Remove espaços ao redor do destino com `strip`.

2. Copia os iteráveis de portas e protocolos para tuplas.

3. Rejeita destino vazio.

4. Rejeita lista vazia, portas não inteiras ou fora de 1 a 65535.

5. Rejeita protocolos vazios ou diferentes dos suportados.

6. Rejeita timeout zero, negativo, infinito ou `NaN`, e verifica sua aceitação por `socket.settimeout` em um socket local sem conexão, fechado ao terminar. Valores finitos grandes demais, como `1e20`, geram `ValueError` de configuração (saída 2 na CLI), em vez de falhar dentro da sondagem. O limite depende do ambiente, sem teto arbitrário no projeto; a verificação não envia tráfego nem altera padrões globais.

7. Salva os valores normalizados e remove repetições.

`type(port) is int` também rejeita `True` e `False`, apesar de os booleanos terem relação com inteiros em Python. `math.isfinite` evita que um valor sem prazo finito seja aceito como timeout.

`tuple(dict.fromkeys(...))` elimina repetições mantendo a ordem de inserção. A cópia para tupla impede que uma lista externa alterada posteriormente modifique silenciosamente a configuração.

`object.__setattr__` é utilizado apenas na normalização inicial: permite guardar os valores finais em uma dataclass congelada. Não use essa técnica para modificar uma configuração durante a execução.

5.3 Como mudar configurações

Crie uma nova configuração ou use `dataclasses.replace`:

```
from dataclasses import replace
from port_scanner.config import ScanConfig

original = ScanConfig('127.0.0.1', ports=(8000,))
modified = replace(original, timeout=0.5)
assert original.timeout == 1.0
assert modified.timeout == 0.5
```

A validação verifica os valores descritos, não a existência do host ou o formato completo de cada nome aceito pelo resolvedor. A resolução ocorre depois, no scanner. Uma falha de codificação do hostname nessa fronteira, como um rótulo ASCII de 64 caracteres, é normalizada de `UnicodeError` para `socket.gaierror`, preservando a causa. A CLI apresenta `Cannot resolve target` e sai com código `1`, sem traceback. Essa captura envolve apenas a chamada ao resolvedor: erros de programação das sondagens, incluindo `ValueError` e `UnicodeError`, continuam propagando.

6. Sondagens: lendo probes.py

6.1 Contrato comum

`probe_tcp` e `probe_udp` recebem `target_ip`, `port` e `timeout`, nessa ordem. Retornam um texto de estado. Não leem o teclado nem imprimem mensagens.

As funções de baixo nível pressupõem entradas válidas. Para uso normal, prefira `scan(ScanConfig(...))`, que separa validação e execução. Uma chamada direta às sondagens deixa a validação sob responsabilidade do chamador.

`with socket.socket(...) as sock` garante que o socket seja fechado ao sair do bloco, tanto no sucesso quanto na exceção. Isso evita consumir recursos do sistema ao testar muitas portas. `sock.settimeout` afeta apenas aquele socket, não o processo inteiro.

6.2 Caminho TCP

A função cria um socket `AF_INET` e `SOCK_STREAM`: família IPv4 e transporte TCP. Depois configura o timeout e chama `connect_ex((target_ip, port))`.

Ao sair do bloco `with`, o socket é fechado. Só então a função traduz o código para o estado. Separar o contato com a rede da sequência de comparações facilita conferir a regra de interpretação.

`errno.errorcode.get(result, result)` procura o nome simbólico de um código. Se ele não estiver mapeado, usa o próprio número, em vez de falhar durante a apresentação do erro.

6.3 Caminho UDP

A função usa `SOCK_DGRAM`. O socket é associado ao par remoto, envia o payload e tenta receber até `UDP_BUFFER_SIZE` bytes, atualmente 4096.

A função retorna `open` quando `recv` conclui. As exceções específicas são tratadas antes dos erros genéricos: `ConnectionRefusedError` indica recusa e `TimeoutError` indica resultado inconclusivo.

Erros de rede que não entram nesses dois casos são propagados. Essa decisão evita espalhar a mesma regra de continuação em cada sondagem: o scanner decide como registrar o erro individual.

7. Orquestração: lendo scanner.py

7.1 Tipos e resultado

`Probe` descreve uma função que recebe texto, inteiro e número e devolve texto. `Resolver` descreve uma função que recebe o destino e devolve um endereço.

`ScanResult` é outra dataclass congelada. Guarda `target_ip`, `port`, `protocol` e `state`. Ela representa uma observação, e não a configuração ou um objeto que controla a rede.

7.2 O gerador scan

`scan` recebe a configuração e, opcionalmente, funções que substituem as dependências de rede. O asterisco na assinatura exige que `resolver` e `probe_functions` sejam informados pelo nome, tornando a chamada mais clara.

Sem substituições, usa `socket.gethostbyname`, `probe_tcp` e `probe_udp`. `target_ip = resolve(config.target)` acontece uma vez antes dos laços. Não é necessário resolver novamente o mesmo destino para cada porta.

O laço externo percorre portas. O interno percorre protocolos. Assim, para portas `(8000, 9999)` e protocolos `('tcp', 'udp')`, a sequência é TCP/8000, UDP/8000, TCP/9999 e UDP/9999.

`yield` entrega um `ScanResult` por vez. Isso torna `scan` um gerador: chamar a função cria um iterador, mas o código interno só começa quando o chamador solicita um item, por exemplo em um `for`.

```
from port_scanner.config import ScanConfig
from port_scanner.scanner import scan

results = scan(ScanConfig('127.0.0.1', ports=(8000,), timeout=0.2))
# A rede ainda nao foi acessada por esse gerador.
for result in results:
    print(result.port, result.state)
```

Não é necessário acumular os resultados em uma lista. Se desejar guardar tudo, use `list(scan(config))`, ciente de que isso executa as sondagens durante a criação da lista.

7.3 Falhas e dependências substituíveis

Um `OSError` gerado dentro da sondagem vira texto `error: ...` no resultado. A execução continua. Falha de DNS ocorre antes desse bloco e é propagada. Erros de programação como `RuntimeError` não são capturados pelo scanner.

A injeção de dependências significa fornecer outra função em vez de fazer o módulo criar ou localizar tudo sozinho. Não exige framework. No teste abaixo, nada é enviado à rede:

```
from port_scanner.config import ScanConfig
from port_scanner.scanner import scan

results = list(scan(
    ScanConfig('exemplo.test', ports=(8000,)),
    resolver=lambda name: '127.0.0.1',
    probe_functions={'tcp': lambda ip, port, timeout: 'closed'},
))
assert results[0].state == 'closed'
```

Esse resultado é controlado pelo exemplo, não uma medição real. O mapa fornecido pelo chamador deve incluir todos os protocolos solicitados e funções que respeitem o contrato. Mapas incompletos são erro do chamador, não indicação de porta fechada.

8. Interface e compatibilidade

8.1 Funções da CLI

`build_parser` cria o parser de `argparse`. Ele define argumentos aceitos, padrões, mensagens de ajuda e escolhas de protocolo. `argparse` trata erros de sintaxe e normalmente encerra com código 2.

`parse_ports` divide o texto por vírgulas e converte cada trecho em inteiro. Texto inválido e portas fora da faixa viram `ArgumentTypeError`, que o parser apresenta ao usuário.

`choose_protocol` usa um dicionário para mapear as opções numéricas para `TCP`, `UDP` e `BOTH`. O laço repete até encontrar uma chave válida. Não é usada recursão: milhares de erros de digitação não acumulam chamadas de função.

`main` lê os argumentos, exibe o aviso, escolhe protocolo e destino, constrói `ScanConfig` e consome `scan`. O cabeçalho com o endereço resolvido é impresso quando chega o primeiro resultado. Cada resultado é apresentado em uma linha.

As exceções previstas pela interface viram códigos de saída. A captura de `ValueError` é restrita à construção de `ScanConfig`: um erro de programação em uma sondagem personalizada não é apresentado como configuração inválida. Existe um teste de regressão específico para preservar essa distinção. Ao estender a interface, restrinja o tratamento à etapa que realmente valida a entrada.

8.2 Arquivos de entrada

O bloco `if __name__ == '__main__'` diferencia executar um arquivo de importá-lo. Importar os módulos não deve iniciar perguntas ou sondagens.

`port_scanner/__main__.py` delega à CLI e utiliza `SystemExit` para devolver o código ao sistema. `main.py` preserva a entrada antiga e injeta as funções oferecidas por `connection.py`.

`connection.py` é uma fachada: exporta `connection_choice`, `connection_tcp` e `connection_udp` como aliases das implementações novas. Isso preserva importações anteriores sem manter duas cópias do algoritmo.

Os atributos `main.PORTS` e `main.TIMEOUT` existem para leitura por código legado, mas não são mais pontos de configuração. Alterá-los não muda o scanner. Use argumentos de linha de comando ou `ScanConfig`.

9. Por que a estrutura facilita manutenção

Responsabilidade única: cada módulo possui uma razão principal para mudar. Alterar mensagens não exige mexer nos sockets; alterar uma sondagem não exige reescrever o menu.

Nomes claros: `ScanConfig` representa parâmetros; `ScanResult` representa observação; `probe_tcp` faz a tentativa TCP. Não é preciso adivinhar o significado de abreviações genéricas.

Uma fonte de verdade: os padrões estão na configuração e as sondagens têm apenas uma implementação. A fachada antiga delega em vez de copiar.

Baixo acoplamento: o scanner não conhece o terminal. Pode ser chamado por testes ou outro programa, e suas dependências podem ser substituídas por funções.

Imutabilidade: parâmetros e resultados não mudam inesperadamente durante uma execução. Isso reduz o conjunto de estados que o leitor precisa acompanhar.

Falhas explícitas: recusa, silêncio e erro de rede são situações diferentes. Transformar tudo em `closed` simplificaria o texto à custa de uma conclusão incorreta.

Simplicidade: não foram introduzidos serviços abstratos, herança, paralelismo ou frameworks para um problema pequeno. Clean code não significa criar camadas sem finalidade.

Há escolhas conscientes: os estados continuam como texto para compatibilidade, e a execução é sequencial para facilitar a leitura. Uma futura necessidade de JSON estruturado ou alto volume pode justificar novas abstrações, acompanhadas de testes e migração.

10. Como testar com confiança

10.1 Comando e organização

```
python -m unittest -v
```

Execute da raiz. O runner encontra `test_connection.py` e os módulos do pacote `tests`. Os testes históricos de `docs/snapshot/` são uma suíte separada e não substituem a suíte atual.

• `tests/test_config.py`: padrões, rejeição de entradas inválidas e proteção contra mutação externa.

• `tests/test_probes.py`: sockets reais em loopback e simulação de erros específicos do sistema.

• `tests/test_scanner.py`: resolução única, ordem, continuação e propagação de falhas.

• `tests/test_cli.py`: menu, argumentos, formatação e códigos de saída, isolando a rede.

• `tests/test_entrypoints.py`: subprocessos reais das duas entradas e modo interativo com sockets locais.

• `test_connection.py`: nomes e comportamento legados.

10.2 Por que misturar testes reais e simulados

Um mock substitui uma dependência por um objeto controlado. É útil para observar chamadas ou produzir timeout sem depender de firewall. Mas testar apenas mocks não prova que um socket real funciona.

Por isso os testes abrem listeners TCP e sockets UDP em `127.0.0.1`. Os servidores usam porta 0 e consultam a porta atribuída com `getsockname`, evitando depender de uma porta fixa livre.

TCP precisa de um listener para provar `open`; uma porta local sem listener permite conferir recusa. UDP usa um servidor que responde e outro que fica silencioso. Erros como timeout TCP e entrega de recusa UDP são simulados onde uma reprodução real dependeria do sistema.

Threads de resposta devem ter prazo, encerramento e transmissão de falhas ao teste principal. Subprocessos devem ter timeout. Nenhum teste deve varrer hosts públicos.

10.3 Um laboratório manual

No primeiro terminal, inicie o servidor didático local:

```
python docs/labs/tcp_server.py
```

No segundo terminal:

```
python -m port_scanner 127.0.0.1 --ports 8000 --timeout 0.2
```

Esse laboratório exige a porta local 8000 disponível. Encerre o servidor com Ctrl+C. A repetição do scan após o encerramento tende a demonstrar recusa local, mas sempre interprete a saída observada, sem pressupor o resultado de outra máquina.

Os testes automatizados usam portas temporárias e são a verificação preferida para desenvolvimento.

11. Receitas para modificar o código

11.1 Trocar uma porta ou o timeout

Para uma execução, prefira `--ports` e `--timeout`. Para mudar o padrão geral, altere as constantes de `config.py`, atualize `tests/test_config.py` e corrija os exemplos da documentação. Não altere a lista exposta por `main.py` esperando reconfigurar o pacote.

11.2 Corrigir um erro

1. Reproduza o problema em um teste pequeno.

2. Execute esse teste e confirme que falha pelo motivo esperado.

3. Mude apenas o necessário na camada responsável.

4. Execute o teste até passar.

5. Execute toda a suíte e só então limpe nomes ou duplicação.

6. Atualize os documentos se um contrato público mudar.

Esse ciclo é conhecido como teste que falha, implementação que passa e refatoração. O teste funciona como proteção contra o retorno do defeito.

11.3 Acrescentar uma opção

Se a opção é de apresentação, comece em `cli.py` e nos testes da CLI. Se altera parâmetros de rede, avalie um campo em `ScanConfig` e sua validação. Não leia argumentos diretamente em `probes.py`.

Ao adicionar uma saída JSON, separe dados de avisos e perguntas. Um teste deve conferir que `stdout` contém somente o JSON nessa modalidade. Não basta serializar os resultados e deixar mensagens humanas misturadas.

11.4 Criar uma sondagem específica

Primeiro escreva um teste com servidor local que reconheça a requisição e forneça uma resposta controlada. Implemente uma função compatível com o contrato `(ip, port, timeout) -> str`.

Para um experimento de biblioteca, injete a nova função em `probe_functions`. Para incorporá-la ao produto, alinhe protocolos suportados, configuração, registro de funções, escolhas da CLI, testes e

documentação. Não adicione uma opção que o scanner não sabe despachar.

11.5 Acrescentar IPv6 ou concorrência

IPv6 altera resolução, família de socket e endereços; não basta mudar a mensagem da ajuda. Concorrência altera ordem, cancelamento, quantidade de sockets e carga de rede. Defina esses contratos antes da implementação e crie testes próprios.

12. PDF, fontes e reprodução

O texto editável deste guia é `docs/GUIA-DO-PROJETO.md`. O gerador é `docs/build_project_guide.py`. Ele lê o Markdown, acrescenta os módulos principais com numeração original e grava um PDF com sumário e marcadores clicáveis.

O arquivo `docs/guia-do-projeto.manifest.json` registra os hashes SHA-256 dos fontes e documentos usados. Um hash é uma identificação do conteúdo: quando o arquivo muda, o hash permite perceber que o PDF precisa ser regenerado. Ele não prova por si só a qualidade do código.

O apêndice reproduz a implementação existente no momento da geração. Se você editar o código depois, regenere o documento antes de compartilhar. A versão histórica permanece em seus arquivos originais.

12.1 Dependências opcionais

```
python -m venv .venv-pdf
.venv-pdf/bin/python -m pip install -r docs/requirements-pdf.txt
.venv-pdf/bin/python -m unittest discover -s docs -p test_documentation_pdf.py -v
.venv-pdf/bin/python docs/build_project_guide.py
```

Esses comandos são para Linux/macOS; no Windows o executável do ambiente fica em `.venv-pdf/Scripts/python.exe`. O scanner não depende dessas bibliotecas.

O gerador usa ReportLab para diagramação e pypdf para verificar a extração de texto e registrar páginas. As fontes precisam estar disponíveis; por padrão, procura Liberation em `/usr/share/fonts/liberation/`. Para outro local, informe `--font-dir` com os arquivos LiberationSans-Regular.ttf, LiberationSans-Bold.ttf e LiberationMono-Regular.ttf.

O gerador escapa caracteres especiais, diferencia títulos, listas e blocos de código, e mantém o separador literal de estados como `open|filtered`. Linhas longas de código podem continuar visualmente com `|`, sem ganhar um número de linha novo. O código original não é reformatado pela geração do PDF.

12.2 Checklist antes de publicar

• Execute a suíte atual completa e os testes opcionais do gerador.

• Gere o PDF novamente depois da última alteração de fonte.

• Confira extração de texto, sumário, acentos, quebras de página e listagens.

• Verifique se os hashes do manifesto correspondem aos fontes atuais.

• Revise o diff e não publique ambientes virtuais, caches, credenciais ou arquivos pessoais.

• Ao enviar a um repositório existente, preserve seu histórico e examine o conteúdo remoto antes de integrar as alterações.

Consulte também `README.md` para referência rápida, `docs/ARQUITETURA.md` para contratos e `CONTRIBUTING.md` para o fluxo de contribuição.

Código-fonte — main.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
  1 """Entrada de script compatível com a versão anterior; delega a port_scanner."""
  2 from collections.abc import Sequence
  3
  4 import connection
  5 from port_scanner.cli import main as run_cli
  6 from port_scanner.config import DEFAULT_PORTS, DEFAULT_TIMEOUT
  7
  8 # Mantidos para importações existentes; use ScanConfig em código novo.
  9 PORTS = list(DEFAULT_PORTS)
 10 TIMEOUT = DEFAULT_TIMEOUT
 11
 12
 13 def main(argv: Sequence[str] | None = None) -> int:
 14     return run_cli(argv, probe_functions={
 15         'tcp': connection.connection_tcp,
 16         'udp': connection.connection_udp,
 17     })
 18
 19
 20 if __name__ == '__main__':
 21     exit_code = main()
 22     if exit_code:
 23         raise SystemExit(exit_code)
```

Código-fonte — connection.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
  1 """Fachada para importações antigas; prefira port_scanner em código novo.
  2
  3 Preserva nomes públicos e assinaturas. O menu pertence à CLI e as operações
  4 de rede ficam em probes. Nenhuma tarefa é iniciada durante a importação.
  5 """
  6 from port_scanner.cli import choose_protocol as connection_choice
  7 from port_scanner.probes import probe_tcp as connection_tcp
  8 from port_scanner.probes import probe_udp as connection_udp
  9
 10 __all__ = ['connection_choice', 'connection_tcp', 'connection_udp']
```

Código-fonte — port_scanner/__init__.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
1 """Scanner IPv4 TCP/UDP sequencial, usando apenas a biblioteca padrão."""
```

Código-fonte — port_scanner/__main__.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
1 """Permite python -m port_scanner sem executar tarefas ao importar o módulo."""
2 from .cli import main
3
4 if __name__ == '__main__':
5     raise SystemExit(main())
```

Código-fonte — port_scanner/config.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
 1 """Configuração imutável e validação, independentes da interface."""
 2 from dataclasses import dataclass
 3 import math
 4 import socket
 5
 6 DEFAULT_PORTS = (53, 22, 80, 111, 139, 443, 32768)
 7 DEFAULT_TIMEOUT = 1.0
 8 SUPPORTED_PROTOCOLS = ('tcp', 'udp')
 9
10
11 @dataclass(frozen=True)
12 class ScanConfig:
13     """Solicitação validada que preserva a ordem de portas e protocolos."""
14
15     target: str
16     ports: tuple[int, ...] = DEFAULT_PORTS
17     protocols: tuple[str, ...] = ('tcp',)
18     timeout: float = DEFAULT_TIMEOUT
19
20     def __post_init__(self) -> None:
21         target = self.target.strip()
22         ports = tuple(self.ports)
23         protocols = tuple(self.protocols)
24         if not target:
25             raise ValueError('Target must not be empty.')
26         if not ports or any(type(port) is not int or not 1 <= port <= 65535 for port in
   | ports):
27             raise ValueError('Ports must be integers between 1 and 65535.')
28         if not protocols or any(protocol not in SUPPORTED_PROTOCOLS for protocol in
   | protocols):
29             raise ValueError('Choose tcp, udp, or both protocols.')
30         if not math.isfinite(self.timeout) or self.timeout <= 0:
31             raise ValueError('Timeout must be a finite number greater than zero.')
32         # Ask the runtime for its supported range without connecting or changing
   | defaults.
33         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
34             try:
35                 sock.settimeout(self.timeout)
36             except (OverflowError, ValueError) as error:
37                 raise ValueError('Timeout is outside the range supported by
   | socket.settimeout.') from error
38         object.__setattr__(self, 'target', target)
39         object.__setattr__(self, 'ports', tuple(dict.fromkeys(ports)))
40         object.__setattr__(self, 'protocols', tuple(dict.fromkeys(protocols)))
```

Código-fonte — port_scanner/probes.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
 1 """Operações de socket IPv4, sem entrada/saída de terminal ou orquestração."""
 2 import errno
 3 import socket
 4
 5 from .config import DEFAULT_TIMEOUT
 6
 7 UDP_PAYLOAD = b'probe'
 8 UDP_BUFFER_SIZE = 4096
 9 TCP_TIMEOUT_ERRORS = frozenset((errno.ETIMEDOUT, errno.EAGAIN, errno.EWOULDBLOCK))
10
11
12 def probe_tcp(target_ip: str, port: int, timeout: float = DEFAULT_TIMEOUT) -> str:
13     """Tenta conectar uma vez. Só uma recusa explícita significa porta fechada.
14
15     Outras exceções de socket são propagadas para o orquestrador.
16     """
17     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
18         sock.settimeout(timeout)
19         result = sock.connect_ex((target_ip, port))
20
21     if result == 0:
22         return 'open'
23     if result == errno.ECONNREFUSED:
24         return 'closed'
25     if result in TCP_TIMEOUT_ERRORS:
26         return 'filtered/unknown'
27     return f'error: {errno.errorcode.get(result, result)}'
28
29
30 def probe_udp(target_ip: str, port: int, timeout: float = DEFAULT_TIMEOUT) -> str:
31     """Envia um datagrama genérico; silêncio não distingue aberto de filtrado.
32
33     UDP connect não faz handshake. O serviço pode ignorar o payload; a entrega
34     de recusa ICMP depende do sistema. Outros OSError são propagados.
35     """
36     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
37         sock.settimeout(timeout)
38         try:
39             sock.connect((target_ip, port))
40             sock.send(UDP_PAYLOAD)
41             sock.recv(UDP_BUFFER_SIZE)
42             return 'open'
43         except ConnectionRefusedError:
44             return 'closed'
45         except TimeoutError:
46             return 'open|filtered'
```

Código-fonte — port_scanner/scanner.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
 1 """Resolve um destino IPv4 e entrega resultados sequenciais de sondagem."""
 2 from collections.abc import Callable, Iterator, Mapping
 3 from dataclasses import dataclass
 4 import socket
 5
 6 from .config import ScanConfig
 7 from .probes import probe_tcp, probe_udp
 8
 9 Probe = Callable[[str, int, float], str]
10 Resolver = Callable[[str], str]
11
12
13 @dataclass(frozen=True)
14 class ScanResult:
15     """Uma observação, não uma garantia de disponibilidade do serviço."""
16
17     target_ip: str
18     port: int
19     protocol: str
20     state: str
21
22
23 def scan(
24     config: ScanConfig,
25     *,
26     resolver: Resolver | None = None,
27     probe_functions: Mapping[str, Probe] | None = None,
28 ) -> Iterator[ScanResult]:
29     """Resolve uma vez e entrega resultados na ordem de portas e protocolos.
30
31     Erros de rede de uma sondagem viram resultados individuais. Falhas de DNS
32     são propagadas. A execução, incluindo DNS, só começa na iteração. As
33     dependências opcionais permitem testes e uso sem acoplamento ao terminal.
34     """
35     resolve = socket.gethostbyname if resolver is None else resolver
36     probes = {'tcp': probe_tcp, 'udp': probe_udp} if probe_functions is None else
   | probe_functions
37     try:
38         target_ip = resolve(config.target)
39     except UnicodeError as error:
40         raise socket.gaierror(socket.EAI_NONAME, f'Invalid hostname encoding: {error}')
   | from error
41     for port in config.ports:
42         for protocol in config.protocols:
43             try:
44                 state = probes[protocol](target_ip, port, config.timeout)
45             except OSError as error:
46                 state = f'error: {error}'
47             yield ScanResult(target_ip, port, protocol, state)
```

Código-fonte — port_scanner/cli.py

Numeração original. Uma continuação visual recebe |, sem um novo número de linha. Use o arquivo original para editar ou executar.

```
 1 """Argumentos, perguntas interativas, apresentação e códigos de saída."""
 2 import argparse
 3 from collections.abc import Mapping, Sequence
 4 import socket
 5 import sys
 6
 7 from .config import DEFAULT_PORTS, DEFAULT_TIMEOUT, ScanConfig
 8 from .scanner import Probe, scan
 9
10 PROTOCOL_GROUPS = {'TCP': ('tcp',), 'UDP': ('udp',), 'BOTH': ('tcp', 'udp')}
11
12
13 def choose_protocol() -> str:
14     """Repete entradas inválidas sem recursão e devolve o rótulo legado."""
15     choices = {'1': 'TCP', '2': 'UDP', '3': 'BOTH'}
16     while True:
17         print('Choose connection type:\n1. TCP\n2. UDP\n3. BOTH')
18         choice = input('Enter your choice (1, 2 or 3): ').strip()
19         if choice in choices:
20             return choices[choice]
21         print('Invalid choice. Please select 1, 2 or 3.')
22
23
24 def parse_ports(value: str) -> tuple[int, ...]:
25     """Lê portas separadas por vírgulas; intervalos não são suportados."""
26     try:
27         ports = tuple(int(item.strip()) for item in value.split(','))
28     except ValueError as error:
29         raise argparse.ArgumentTypeError('Ports must be comma-separated integers.') from
   | error
30     if any(not 1 <= port <= 65535 for port in ports):
31         raise argparse.ArgumentTypeError('Ports must be between 1 and 65535.')
32     return ports
33
34
35 def build_parser() -> argparse.ArgumentParser:
36     parser = argparse.ArgumentParser(description='Sequential IPv4 TCP/UDP port scanner.')
37     parser.add_argument('target', nargs='?', help='IPv4 address or hostname; omit for
   | interactive mode')
38     parser.add_argument('--protocol', choices=('tcp', 'udp', 'both'),
39                         help='default: tcp with a target; otherwise ask interactively')
40     parser.add_argument('--ports', type=parse_ports, default=DEFAULT_PORTS,
41                         help='comma-separated ports (default: %(default)s)')
42     parser.add_argument('--timeout', type=float, default=DEFAULT_TIMEOUT,
43                         help='per-socket timeout in seconds (default: %(default)s)')
44     return parser
45
46
47 def main(argv: Sequence[str] | None = None, *, probe_functions: Mapping[str, Probe] |
   | None = None) -> int:
48     """Executa a interface; argparse trata erros de sintaxe com SystemExit(2)."""
49     args = build_parser().parse_args(argv)
50     try:
51         print('Scan only hosts you own or have permission to test.')
52         protocol = args.protocol.upper() if args.protocol else ('TCP' if args.target is
   | not None else choose_protocol())
53         target = args.target if args.target is not None else input('Insert the target IP
   | or hostname: ')
54         try:
55             config = ScanConfig(target, args.ports, PROTOCOL_GROUPS[protocol],
   | args.timeout)
```

```
        except ValueError as error:
            print(f'Invalid configuration: {error}', file=sys.stderr)
            return 2
        first_result = True
        for result in scan(config, probe_functions=probe_functions):
            if first_result:
                print('-' * 50)
                print(f'Scanning {result.target_ip} ({protocol})...')
                print('-' * 50)
                first_result = False
            print(f'{result.port}/{result.protocol}: {result.state}')
    except socket.gaierror as error:
        print(f'Cannot resolve target: {error}', file=sys.stderr)
        return 1
    except (KeyboardInterrupt, EOFError):
        print('\nProgram closed by the user!', file=sys.stderr)
        return 130
    return 0
```